AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 23: Matrix Factorisation

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Matrix completion, low-rank approximation, recommender systems, alternating least squares, non-negative matrix factorisation

2 Main Points

2.1 Missing Data and Recommenders

- We are often given a matrix of data with missing entries (e.g. forms that people have only partly filled in)
  - most learning machines need complete data, so we either throw away rows or columns with missing values or try to infer them
- We can only infer missing values if the matrix has structure. The structure we use is that *structured data typically lies close to a low-dimensional subspace* (the idea behind PCA and the SVD)
- The best-known application is *collaborative filtering*: predicting the ratings users would give to items they haven't rated, from the ratings of all users
  - the classic method finds similar users and averages their ratings, weighted by similarity
    $$\hat{R}_{ui} = \frac{\sum_{u'} \mathrm{sim}(u,u')\, R_{u'i}}{\sum_{u'} \mathrm{sim}(u,u')}$$
  - the simplest baseline uses means: $R_{ui} \approx \bar{R}_u + \bar{R}_i - \bar{R}$ (user mean plus item mean minus overall mean)

2.2 Low-Rank Approximation

- The SVD of a complete $m \times n$ matrix is
  $$\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T} = \sum_k s_k\, \boldsymbol{u}_k\, \boldsymbol{v}_k^\mathsf{T}, \qquad s_1 \geq s_2 \geq \cdots \geq 0$$
- Keeping the K largest singular values gives the rank-K matrix $\mathbf{X}_K = \sum_{k=1}^{K} s_k\, \boldsymbol{u}_k\, \boldsymbol{v}_k^\mathsf{T}$
- *Eckart–Young theorem*: $\mathbf{X}_K$ is the best rank-K approximation to $\mathbf{X}$
  - in the Frobenius norm, with $\|\mathbf{X} - \mathbf{X}_K\|_F^2 = \sum_{k>K} s_k^2$
  - and in the spectral norm, with $\|\mathbf{X} - \mathbf{X}_K\|_2 = s_{K+1}$
  - if the data lies close to a K-dimensional subspace, the discarded singular values are small

- **Completing a matrix with the SVD.** The SVD needs a complete matrix, so we iterate

  1. fill the missing entries (e.g. with user means)
  2. compute the best rank-K approximation $\mathbf{L}(t)$ of the filled-in matrix $\mathbf{X}(t)$
  3. copy its values into the missing entries only

  $$\mathbf{X}(t+1) = \mathbf{W} \odot \mathbf{X} + \overline{\mathbf{W}} \odot \mathbf{L}(t)$$

  where $W_{ij} = 1$ if X_{ij} is observed and 0 otherwise, $\overline{\mathbf{W}} = 1 - \mathbf{W}$ and $\odot$ is the element-by-element product
  4. repeat from 2
  5. in the lecture's example (a rank-2 matrix with a quarter of its entries hidden) the RMS error on the hidden entries falls from 0.95 (user means) to 0.22 after four iterations

- More directly, write $\mathbf{X} \approx \mathbf{A}\,\mathbf{B}$ with $\mathbf{A}$ an $m \times K$ and $\mathbf{B}$ a $K \times n$ matrix
  - $\mathbf{AB}$ has rank at most K, and has $(m+n)K$ parameters rather than mn
  - K controls the complexity; choose it by cross-validation, hiding some of the observed entries
  - fit by least squares on the observed entries $\mathcal{D}$

  $$\min_{\mathbf{A},\mathbf{B}} \sum_{(i,j)\in\mathcal{D}} \Big(X_{ij} - \sum_{k=1}^{K} A_{ik}\, B_{kj}\Big)^2$$

  - **LoRA.** The same parameter count makes fine-tuning large neural networks cheap. Low-rank adaptation (LoRA; Hu et al.,

    1. freezes a pretrained weight matrix $\mathbf{W}_0$ ($m \times n$)

    and learns only a low-rank change $\Delta\mathbf{W} = \mathbf{AB}$ with $K \ll m, n$, one factor starting at zero so that training starts from $\mathbf{W}_0$. For $m = n = 4096$ and $K = 8$ that is about 6.6×10^4 trainable parameters instead of 1.7×10^7. The assumption is that the change a new task needs is close to low rank. Each task then stores only its own small $\mathbf{A}$ and $\mathbf{B}$, and after training $\mathbf{AB}$ can be added into $\mathbf{W}_0$, so the adapted network is no slower to run
  - $\mathbf{A}$ and $\mathbf{B}$ are never unique: $\mathbf{AB} = (\mathbf{AG})(\mathbf{G}^{-1}\mathbf{B})$ for any invertible $K \times K$ matrix $\mathbf{G}$

2.3 Probabilistic Approach

- We use the *maximum a posteriori* (MAP) solution from the lecture on probabilistic inference: the most probable parameters, $\boldsymbol{\theta}_{\mathrm{MAP}} = \mathrm{argmax}_{\boldsymbol{\theta}} \big(\log \mathbb{P}[\mathcal{D}|\boldsymbol{\theta}] + \log \mathbb{P}[\boldsymbol{\theta}]\big)$, which does not need the evidence. Its limitations (no error bars, dependence on how we parametrise $\boldsymbol{\theta}$) are discussed there

- Treat the ratings as noisy: $X_{ij} \sim \mathcal{N}\big(\sum_k A_{ik}B_{kj},\, \sigma^2\big)$, with Gaussian priors $A_{ik} \sim \mathcal{N}(0, \sigma_A^2)$ and $B_{kj} \sim \mathcal{N}(0, \sigma_B^2)$

- The full posterior is intractable, but the *maximum a posteriori* (MAP) solution maximises

$$\mathcal{E} = -\frac{1}{2\sigma^2} \sum_{(i,j)\in\mathcal{D}} \Big(X_{ij} - \sum_{k=1}^{K} A_{ik}\, B_{kj}\Big)^2 - \frac{1}{2\sigma_A^2} \sum_{i,k} A_{ik}^2 - \frac{1}{2\sigma_B^2} \sum_{k,j} B_{kj}^2 + \text{const}$$

- **Each step is ridge regression.** Fix $\mathbf{B}$ and write $\boldsymbol{a}_i$ for row i of $\mathbf{A}$ and $\boldsymbol{b}_j$ for column j of $\mathbf{B}$. Then $\boldsymbol{a}_i$ minimises

$$\sum_{j|(i,j)\in\mathcal{D}} \big(X_{ij} - \boldsymbol{a}_i^{\mathsf{T}}\boldsymbol{b}_j\big)^2 + \eta\, \|\boldsymbol{a}_i\|^2, \qquad \eta = \frac{\sigma^2}{\sigma_A^2}$$

which is ridge regression with features $\boldsymbol{b}_j$ and targets X_{ij}. Its solution is

$$\boldsymbol{a}_i = \left(\eta\,\mathbf{I} + \sum_j \boldsymbol{b}_j \boldsymbol{b}_j^{\mathsf{T}}\right)^{-1} \sum_j \boldsymbol{b}_j\, X_{ij}$$

(sums over the j with $(i, j) \in \mathcal{D}$): one $K \times K$ linear solve per row

- Solving for $\mathbf{B}$ with $\mathbf{A}$ fixed is the same with the roles swapped. Alternating the two is *alternating least squares* (ALS)
  - each step is a convex problem and cannot decrease $\mathcal{E}$, so ALS converges, usually quickly
  - but the whole problem is not convex, so it converges to a local optimum that can depend on the starting point
- **Embeddings.** Row i of $\mathbf{A}$ is a K-dimensional vector for user i and column j of $\mathbf{B}$ a vector for item j; the predicted rating is their inner product. Modern recommenders learn such embeddings with neural networks, mostly from *implicit feedback* (clicks, views, purchases), where a missing entry is weak evidence of not liking rather than simply unknown
- Matrix factorisation was central to the winning entries of the Netflix Prize (2006–09) and remains a strong baseline

2.4 Non-Negative Matrix Factorisation

- Factorisation is also a way of modelling data. For a words $\times$ books count matrix, the rank-3 SVD gives factors with mixed signs, which are hard to interpret
- *Non-negative matrix factorisation* (NMF) minimises $\|\mathbf{X} - \mathbf{AB}\|_F^2$ with every element of $\mathbf{A}$ and $\mathbf{B}$ non-negative
  - projected gradient descent: take a gradient step, then set negative elements to zero
  - or Lee and Seung's multiplicative updates ($\oslash$ is element-by-element division)

    $$\mathbf{B} \leftarrow \mathbf{B} \odot \left(\mathbf{A}^{\mathsf{T}}\mathbf{X}\right) \oslash \left(\mathbf{A}^{\mathsf{T}}\mathbf{AB}\right), \qquad \mathbf{A} \leftarrow \mathbf{A} \odot \left(\mathbf{XB}^{\mathsf{T}}\right) \oslash \left(\mathbf{ABB}^{\mathsf{T}}\right)$$

    which never make an element negative
  - in the lecture's example the three factors pick out mathematics, physics and biology
- Caveats
  - NMF cannot fit better than the truncated SVD, which is the best rank-K fit (Eckart–Young). In the example the relative errors are 0.164 (NMF) and 0.158 (SVD)
  - the problem is not convex, so the result depends on the starting point and on K
  - the factors are not unique, and a "parts-based" interpretation of them is not guaranteed
  - so compare against simple baselines such as the SVD or PCA

3 Exercises

3.1 Counting parameters

- A recommender has $m = 10^6$ users and $n = 10^5$ items. How many numbers does a rank $K = 50$ factorisation store, compared with the full matrix?

3.2 Eckart–Young

- A matrix has singular values 5, 3 and 1. What are the Frobenius-norm and spectral-norm errors of its best rank-1 and rank-2 approximations?

3.3 The prior fixes the scale

- Show that replacing $\mathbf{A} \to c\,\mathbf{A}$ and $\mathbf{B} \to \mathbf{B}/c$ (for $c > 0$) leaves the likelihood unchanged.
- Show that the prior terms in $\mathcal{E}$ are then $-\alpha\, c^2 - \beta/c^2$ with $\alpha = \sum_{i,k} A_{ik}^2/(2\sigma_A^2)$ and $\beta = \sum_{k,j} B_{kj}^2/(2\sigma_B^2)$, and find the c that maximises them. What does this say about the sizes of $\mathbf{A}$ and $\mathbf{B}$ at the MAP solution?

3.4 Ridge regression for one row

- Differentiate $\sum_j (X_{ij} - \boldsymbol{a}_i^{\mathsf{T}} \boldsymbol{b}_j)^2 + \eta\, \|\boldsymbol{a}_i\|^2$ with respect to $\boldsymbol{a}_i$ and derive the update for $\boldsymbol{a}_i$ given above.

3.5 Multiplicative updates and KKT

- Show that if every element of $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{X}$ is positive, the multiplicative update for $\mathbf{B}$ keeps $\mathbf{B}$ positive.
- At a fixed point of the update show that for every element either $B_{kj} = 0$ or $\big(\mathbf{A}^{\mathsf{T}}\mathbf{A}\mathbf{B} - \mathbf{A}^{\mathsf{T}}\mathbf{X}\big)_{kj} = 0$. Relate this to the complementary slackness condition in the KKT conditions for minimising E subject to $B_{kj} \geq 0$ (the gradient of E with respect to $\mathbf{B}$ is $2(\mathbf{A}^{\mathsf{T}}\mathbf{A}\mathbf{B} - \mathbf{A}^{\mathsf{T}}\mathbf{X})$).

4 Experiments

4.1 Alternating least squares

- Make a noisy rank-3 matrix, observe 30% of its entries, hold some of those back, and fit by ALS for several K and η

```
import numpy as np

rng = np.random.default_rng(1)
m, n, K_true = 60, 40, 3
X_true = rng.standard_normal((m, K_true)) @ rng.standard_normal((K_true, n))
X = X_true + 0.3 * rng.standard_normal((m, n))     # noisy ratings
observed = rng.random((m, n)) < 0.3                  # see 30% of entries
train = observed & (rng.random((m, n)) < 0.8)        # hold some back
test = observed & ~train


def als(X, W, K, eta, iters=50):
    """Alternating least squares: each step is ridge regression."""
    A = 0.1 * rng.standard_normal((X.shape[0], K))
    B = 0.1 * rng.standard_normal((K, X.shape[1]))
    for _ in range(iters):
        for i in range(X.shape[0]):           # row i of A, B fixed
            Bi = B[:, W[i]]
            A[i] = np.linalg.solve(eta * np.eye(K) + Bi @ Bi.T, Bi @ X[i, W[i]])
        for j in range(X.shape[1]):           # column j of B, A fixed
            Aj = A[W[:, j]]
```

```
            B[:, j] = np.linalg.solve(eta * np.eye(K) + Aj.T @ Aj, Aj.T @ X[W[:, j], j])
    return A, B


def rmse(P, mask):
    return np.sqrt(np.mean((P - X)[mask] ** 2))


mean = X[train].mean()
print(f"predict the mean:  test RMSE {rmse(np.full_like(X, mean), test):.3f}")
for K in [1, 2, 3, 5, 10]:
    for eta in [0.01, 1.0]:
        A, B = als(X, train, K, eta)
        print(f"K={K:2d} eta={eta:4}:  train {rmse(A @ B, train):.3f}  test {rmse(A @ B, te
```

- What happens to the training and test errors as K grows? What does the regulariser η do? (See the answers.)

5 Answers

5.1 Counting parameters

- $(m+n)K = 1.1 \times 10^6 \times 50 = 5.5 \times 10^7$ numbers, against $mn = 10^{11}$ for the full matrix: about 1800 times fewer

5.2 Eckart–Young

- Rank 1: Frobenius error $\sqrt{3^2+1^2} = \sqrt{10} \approx 3.16$, spectral error 3
- Rank 2: Frobenius error 1, spectral error 1

5.3 The prior fixes the scale

- $(c\mathbf{A})(\mathbf{B}/c) = \mathbf{AB}$, so the likelihood term is unchanged
- The prior terms become $-\alpha c^2 - \beta/c^2$. Setting the derivative $-2\alpha c + 2\beta/c^3$ to zero gives $c^4 = \beta/\alpha$
- At the MAP solution $c = 1$, so $\alpha = \beta$: the prior balances the sizes, $\sum A_{ik}^2/\sigma_A^2 = \sum B_{kj}^2/\sigma_B^2$. The prior removes the scaling freedom, but not rotations: $\mathbf{A} \to \mathbf{AQ}$, $\mathbf{B} \to \mathbf{Q}^\mathsf{T}\mathbf{B}$ with $\mathbf{Q}$ orthogonal changes neither $\mathbf{AB}$ nor the sums of squares

5.4 Ridge regression for one row

- The gradient is $-2\sum_j \boldsymbol{b}_j(X_{ij} - \boldsymbol{b}_j^\mathsf{T}\boldsymbol{a}_i) + 2\eta\,\boldsymbol{a}_i$. Setting it to zero, $\big(\eta\,\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\big)\boldsymbol{a}_i = \sum_j \boldsymbol{b}_j X_{ij}$, which is $\mathbf{M}^i\boldsymbol{a}^i = \boldsymbol{c}^i$ in the lecture

5.5 Multiplicative updates and KKT

- Every element of $\mathbf{A}^\mathsf{T}\mathbf{X}$ and $\mathbf{A}^\mathsf{T}\mathbf{AB}$ is a sum of products of positive numbers, so positive. The update multiplies each B_{kj} by the positive ratio of the two, so B_{kj} stays positive
- At a fixed point $B_{kj} = B_{kj}\,(\mathbf{A}^\mathsf{T}\mathbf{X})_{kj}/(\mathbf{A}^\mathsf{T}\mathbf{AB})_{kj}$, so $B_{kj}\big((\mathbf{A}^\mathsf{T}\mathbf{AB})_{kj} - (\mathbf{A}^\mathsf{T}\mathbf{X})_{kj}\big) = 0$
- This is $B_{kj}\,\partial E/\partial B_{kj} = 0$: either the constraint is active ($B_{kj} = 0$) or the gradient component vanishes, as complementary slackness requires. (A full KKT point also needs $\partial E/\partial B_{kj} \geq 0$ where $B_{kj} = 0$, which the fixed-point condition alone does not guarantee)

5.6 Alternating least squares

- Running the code gives (test RMSE; predicting the mean gives 1.67)

K	$\eta = 0.01$	$\eta = 1$
1	1.163	1.185
2	1.116	1.075
3	4.324	0.695
5	2.286	0.898
10	1.884	0.947

- The training error falls steadily with K (to 0.006 for $K = 10$, $\eta = 0.01$), but the test error is best at the true rank, $K = 3$, with regularisation
- Without regularisation the larger models over-fit badly: there are only about 10 training entries per row and 14 per column to fit K parameters each
- The test error cannot reach the noise level (0.3) because so few entries are observed